M4 Beta Build - Helping Hand

Kali Schuchhardt, Gael Garcia, Brian Paz, Srinitha Srikanth

November 13, 2026

Github Repository: `https://github.com/GG1627/helping-hand`

1 Beta Build Overview

Helping Hand is a wearable IOT glove intended to help people learn ASL (American Sign Language). Our project consists of an ESP32, 5 flex sensors, a 6-axis IMU, and an accompanying Flutter mobile application that runs on Android. Our sensors send finger flex and IMU axis data via BLE (Bluetooth Low Energy) from the ESP32 to our mobile application so users can view live telemetry data and practice signing using guided lessons.

Historically, with the Alpha Build, users are able to practice ASL letters A-Z and number 0-9 with our application and glove. Our focus with the Beta Build was to introduce TCN for our ML to recognize signs such as "Hello", "Please", and "Yes". TCN is an ML model that is designed to handle sequential data, as our previous model for the Alpha Build could only handle static data. We also wanted our users to have a way to sign in and be able to save their progress within the cloud across devices, so we added a Firebase backend and created sign in/sign out and password reset workflows with Firebase Authentication. We also added live sign recordings, so we could record and save our signs as CSV/JSON files to refine our ML model to better recognize signs. Lastly, we wanted to revamp our UI and rebrand our app, so we chose a color scheme that we believed encourages learning and designed our UI to be as minimalist as possible so users can focus on signing and learning.

2 Architecture

Our glove includes 5 flex sensors, one per finger, which measure finger bend through voltage-divider circuits. The IMU supplies motion telemetry for display and recording, but its measurements are not used by the static classifier model. It is used by the TCN model for word recognition. The ESP32 runs the flex sensor values through a static MLP classifier and TCN model and sends the prediction and confidence score through BLE (Bluetooth Low Energy) to the Android app. The user selects what they would like to learn (numbers, letters, or words) in the app and attempts the sign. In order for the sign to be marked as learned in the database, the prediction must match the selected target sign at a confidence score of at least 80%, maintained for at least 750 miliseconds and five packets.

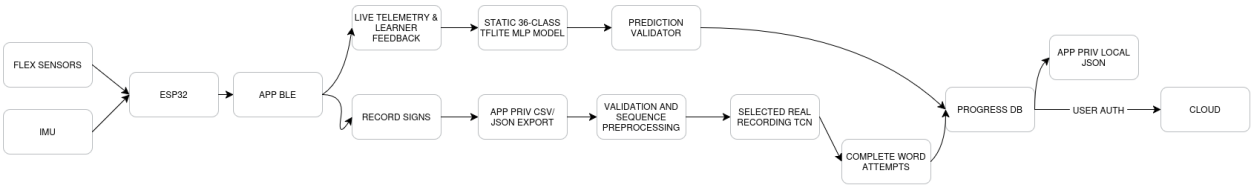
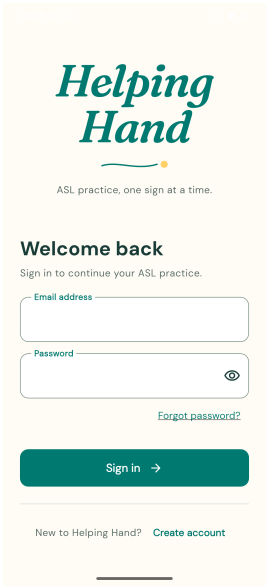


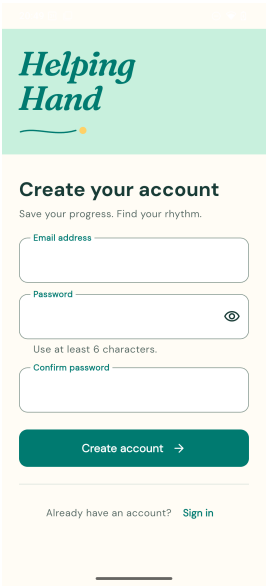
Figure 1: Architecture Flowchart

3 Usability and Feedback

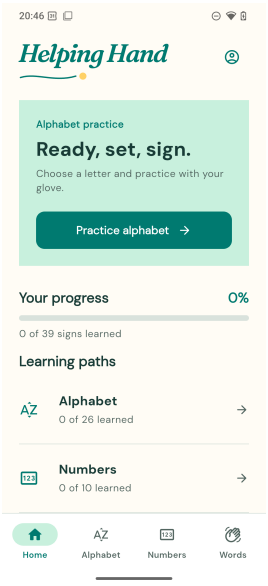
Interface & Navigation: Our app contains a start (sign in) page, a create an account page, a dashboard showing the user's overall learning progress, an Alphabet tab with A-Z targets, a Numbers tab with targets 0-9, a Words tab with lessons on how to learn how to sign words, and a BLE testing page for developers. The BLE testing page provides glove connection controls and displays live telemetry data, including raw and normalized flex sensor values. The record signs on the BLE testing page provides controls for recording labeled sensor trials, reviewing them, and saving or exporting the recordings. The recording signs functionality is for us, the developers, to save our CSV/JSON files in order to train our model to better recognize signs. The practice and progress controls on the practice pages now include retry and reset options just in case a user messes up.



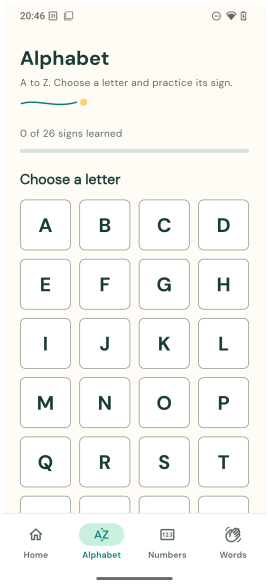
Start Screen



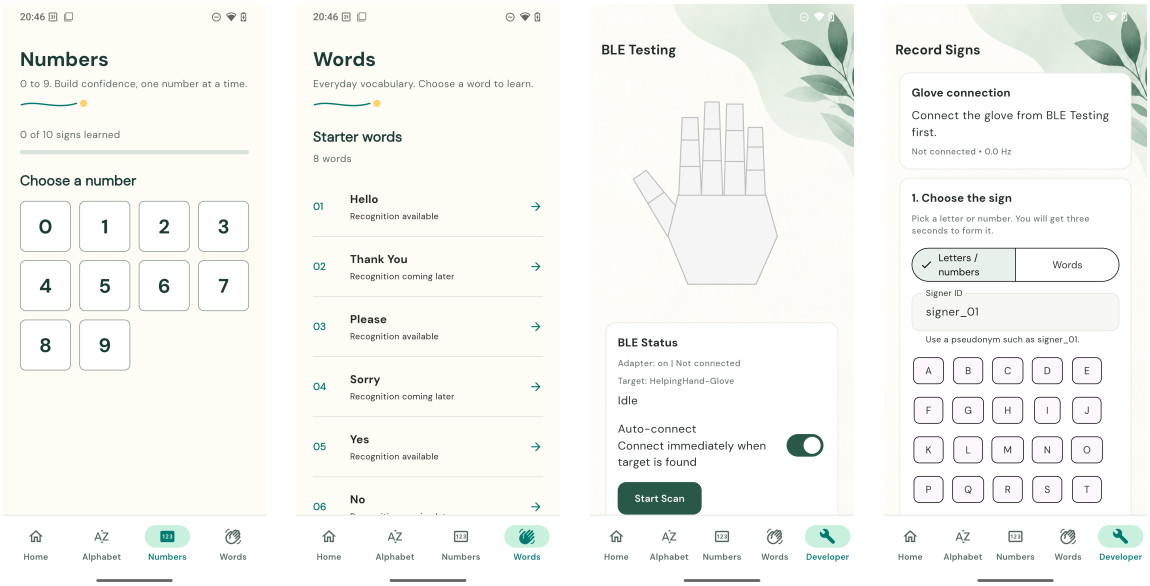
Create Account



Dashboard



Alphabet Tab



Numbers Tab | Words Tab | BLE DEV Screen | Record Signs

Figure 2: Mobile App Screens

Perception: During practice, users can see the current sign prediction, confidence score, and bluetooth connection state. A hold-progress bar shows how long the matching prediction has been maintained. Once a letter or number is learned, its tile changes to a darker color to indicate to the user. The app also displays storage and synchronization statuses so the users can tell whether progress is being saved locally to their device, syncing, synced, or experiencing a failure.

Responsiveness: Our BLE and Firebase operations are ran asynchronously so the app can remain usable while waiting for responses. Progress is stored locally before cloud synchronization, allowing for local progress access while Firebase is unavailable.

4 Build Quality & Validation

We validated our project mainly by software tests and physical tests. We ran a few tests/commands to ensure our software and firmware had no issues or bugs that we did not already know about.

- flutter analyze: Passed with no issues found.
- flutter test: Passed all 16 tests.
- flutter build apk –debug: Passed with a debug APK produced.
- platformio run (in the ESP32 file): Passwd for the ESP32
- ESP32 Memory Report: 37.6% RAM usage and 83.9% app flash usage.
- Ran pytest from backend/tests: Passed 23 tests with 3 TFLite deprecation warnings.

We ensured that the flex sensor and IMU data was reaching our android phone over BLE and tested out signing the alphabet, numbers, the new words we added ("Hello", "Yes", and "Please") while connected to the mobile app. We observed correct predictions for the signs tested and saw tiles change appearance in the app, showing that they were completed and learned. We tested our new recording words functionality and exported those recordings, ensuring that they delievered valid CSV/JSON files. We also tested out our new sign in/sign out functionality and tested out creating new accounts as well. Finally we made sure that our blender model shows the correct model for the correct sign.

Robustness: Our app saves progress locally for the user before cloud synchronization, if there are any failures it reports it and shows the user. Attempts to sign check the incoming data and is canceled when the glove is disconnected via bluetooth, when the user leaves the practice pages, or when the app is closed.

Consistency: Static Sign (Alphabet & Numbers) completion requires the user to match the targeted sign with a prediction confidence score of at least 80%, maintained for at least 750 milliseconds across at least 5 packets. In order for a word to be marked as complete, it requires the user to match the selected word with the highest-scoring prediction with at least 80% confidence. These only define the learning completion requirements and do not represent measured recognition accuracy.

Aesthetic Rigor & Validation: Our app shares the same branding, color scheme, typography, and matching controls all across the board.

5 Bugs & Limitations

Here is a list of known bugs and incomplete features as of the end of our Beta Build:

- Our main.cpp file waits for the USB Serial interface without a timeout. If no serial host opens the port, firmware setup becomes blocked before normal BLE/Sensor initialization.
- The selected letter or number to learn remains active after leaving the practice page. The app can continue accepting matching packets and can record completion while the user is back on the learning tab page.
- On the words tab, only "Hello", "Please", and "Yes" are trained in the TCN model. The rest of the words are just placeholders for now.
- Real time specific ASL blender model is not finished. The practice pages have the Targeted blender model but not an animation of the user's hand in real time.
- J&Z are excluded from our static model.

  Our biggest limitations are that the flex sensors only measure the fingers' bend, while the IMU on the wrist provides motion and orientation information. These do not reliably determine the hand position relative to the body. For example, similar movements performed near the head or the stomach may be difficult for our model to distinguish. We plan to fix this by creating a glove initialization sequence when the user starts the

app up so the IMU knows where the hand is relative to the user's body. We plan to implement this by having the user place their gloved hand in certain areas of the body, such as heart, stomach, and head and record and save those values per session.

Additionally, we have no sensors that are able to detect gaps between fingers. So, for example, U and V cannot be distinguished well with our model. In order to fix this we would need more IMU's and an I2C multiplexer, which is not in the scope of our project. We hope that users would recognize which sign to perform and learn based of our blender model.

6 Remaining Work

At the Beta Milestone, the following work remained for our team to complete:

- Develop the custom PCB and improve sensor attachment and wiring via sewn in conductive thread and layers of insulating fabric.
- Update the hardware schematic with our specific IMU and sensors.
- Train the ML Model further with new words to develop a larger recognition of ASL signs.
- Plan placement of piezoelectric sensors for haptic feedback.
- Develop firmware for piezoelectric haptic feedback using PWM.
- Finalize blender hand models for learning animations and real time animations.
- Set up glove initialization sequence.